

An Empirical Study on Parallelization of Classical Machine Learning Methods

* EE 451 Distributed and Parallel Computing - Final Project Proposal

Eric Huang, Harry Yang, Jinglu Sun, Mo Jiang, and Pinru Wang
University of Southern California
{ehuang97, yangyiha, jinglusu, mojiang, pinruwan}@usc.edu

Abstract—This project presents a from-scratch implementation of five classical machine learning algorithms—K-Nearest Neighbors, Support Vector Machine, Naïve Bayes, Decision Tree, and Multilayer Perceptron—in pure C++17, parallelized using OpenMP and POSIX threads to maximize multi-core CPU utilization on a binary classification task. We conduct a rigorous empirical and analytical benchmarking study on the CS:GO Round Winner Classification dataset [1], investigating how each algorithm’s arithmetic intensity and data dependency structure governs its parallel speedup, efficiency, and classification performance as a function of core count.

Index Terms—parallel computing, multi-core optimization, machine learning, OpenMP, pthreads, K-Nearest Neighbors, Support Vector Machine, Decision Tree, Multilayer Perceptron, Naïve Bayes, arithmetic intensity, speedup analysis

I. INTRODUCTION

A. Background and Hypothesis

Modern machine learning frameworks such as scikit-learn and PyTorch provide high-level abstractions that, while convenient, obscure the underlying hardware utilization and introduce substantial runtime overhead [2], [3]. Accelerating classical ML algorithms from first principles in a multi-core environment is a non-trivial problem because each algorithm exhibits fundamentally different computational characteristics and data dependency structures [4]. For instance, Support Vector Machines (SVM) require iterative optimization procedures with globally shared state [5], and Multilayer Perceptrons (MLP) involve dense matrix operations whose parallel efficiency is tightly coupled to cache locality [6]. These varied bottlenecks make it analytically interesting and practically challenging to reason about how each algorithm responds to an increasing number of CPU cores. Consequently, a detailed performance analysis of these algorithms is essential for identifying the precise ceiling of architectural scalability and informing the design of next-generation, hardware-aware optimization strategies.

We hypothesize that while KNN and Naïve Bayes will achieve high initial speedup before hitting memory bandwidth limits, and Decision Trees and SVM will be constrained by synchronization overhead and iterative convergence, MLP will scale most effectively due to its high arithmetic intensity and suitability for SIMD optimizations.

B. Contributions

Building on previous research into multi-core machine learning optimization [4], this project delivers the following concrete contributions. First, we implement five ML classification algorithms—K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Naïve Bayes (NB), Decision Tree (DT), and Multilayer Perceptron (MLP)—entirely in pure C++17 with no external ML dependencies to examine how specific algorithmic structures influence parallel acceleration. Second, each implementation is parallelized using OpenMP and POSIX threads (pthreads), targeting multi-core CPU architectures with architecture-specific compiler optimizations. Third, we conduct a rigorous empirical and mathematical analysis of each algorithm’s Arithmetic Intensity (floating-point operations per byte of memory traffic), quantifying the degree to which each model is compute-bound versus memory-bound and explaining observed scaling behavior. Finally, we provide an integrated benchmarking engine that evaluates all five models on a shared dataset to analyze how parallelization strategies and accuracy trade-offs diverge across different algorithmic frameworks and implementation constraints.

II. DESCRIPTION AND TECHNICAL APPROACH

A. Parallelization Techniques

We employ two complementary parallelization strategies across our implementations. For data-parallel workloads, we partition the training dataset across available threads so that each thread operates on an independent data partition. This strategy applies directly to KNN distance metric computation and Naïve Bayes class conditional frequency counting, where no inter-thread communication is required during the main computation phase. For task-parallel workloads, we use OpenMP `#pragma omp parallel` for directives to distribute independent subtasks such as feature-split evaluation in the Decision Tree and independent weight gradient updates in the MLP. Where shared mutable state is unavoidable—particularly in SVM’s global support vector updates—we employ fine-grained locking via pthreads mutexes to manage concurrency safely. A central analytical focus of this project is characterizing each algorithm’s Arithmetic Intensity, defined as the ratio of floating-point operations to bytes of memory traffic. This metric determines whether an algorithm is

fundamentally compute-bound or memory-bound and directly predicts the practical ceiling of multi-core speedup, since memory-bound algorithms saturate at the memory bandwidth limit regardless of core count.

B. Implementation Platform

All implementations are written in pure C++17 with no external machine learning libraries. Parallelism is managed through OpenMP for high-level loop-level parallelism and POSIX pthreads for fine-grained threading control. All code is compiled with GCC or Clang using the `-O3` and `-march=native` flags to enable auto-vectorization and hardware-specific instruction selection. Experiments will be conducted on a shared multi-core machine to allow systematic scaling analysis across varying core counts.

III. EVALUATION AND BENCHMARKING

A. Dataset and Metrics

We evaluate all five models on the CS:GO Round Winner Classification dataset [1], which contains 122,411 round snapshots derived from approximately 700 professional tournament matches (2019–2020). Each record consists of 96 pre-processed features spanning team health, economy, equipment, and positional state, with a binary target variable indicating round winner (CT or T). The community benchmark accuracy for this dataset ranges from 76% to 80% under the 80-20 train-test split setting, providing a meaningful baseline for validating model correctness. We report both model quality metrics—accuracy, precision, recall, and F1-score—and system performance metrics including inference latency (ms), throughput (snapshots per second), parallel speedup $S = T_{\text{parallel}}/T_{\text{serial}}$, and Arithmetic Density (Ops/byte) measured per model.

B. Deliverables

The code deliverables consist of the following source files: `knn.cpp`, `svm.cpp`, `nb.cpp`, `dt.cpp`, and `mlp.cpp`, each containing a self-contained parallelized implementation of the respective algorithm; and `analytics_engine.cpp`, a master benchmarking driver responsible for loading the CS:GO dataset, executing all five models under varying core configurations, and writing performance results to structured CSV files for analysis. A final written report will document the mathematical basis for each parallelization strategy, the empirical speedup curves as a function of core count, and a comparative discussion of why certain algorithms benefit more from multi-core execution than others.

REFERENCES

- [1] C. Lillelund, “CS:GO Round Winner Classification.” <https://www.kaggle.com/datasets/christianlillelund/csgo-round-winner-classification/data>, 2020. Accessed: 2025.
- [2] A. Phani, E. Strauss, and S. Schelter, “stratum: A system infrastructure for massive agent-centric ml workloads,” 2026.
- [3] L. Buitinck, G. Louppe, M. Blondel, F. Pedregosa, A. Mueller, O. Grisel, V. Niculae, P. Prettenhofer, A. Gramfort, J. Grobler, R. Layton, J. Vanderplas, A. Joly, B. Holt, and G. Varoquaux, “Api design for machine learning software: experiences from the scikit-learn project,” 2013.
- [4] M. S. Serpa, A. M. Krause, E. H. Cruz, P. O. A. Navaux, M. Pasin, and P. Felber, “Optimizing machine learning algorithms on multi-core and many-core architectures using thread and data mapping,” in *2018 26th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, pp. 329–333, 2018.
- [5] R. Guido, S. Ferrisi, D. Lofaro, and D. Conforti, “An overview on the advancements of support vector machine models in healthcare applications: A review,” *Information*, vol. 15, no. 4, 2024.
- [6] G. Vasiliadis, R. Tsirbas, and S. Ioannidis, “The best of many worlds: Scheduling machine learning inference on cpu-gpu integrated architectures,” in *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 55–64, 2022.